MTH2005
Case Study 2

Jonas Brown

Introduction

Throughout this report, I will be answering the questions provided in the brief using theory and numerical methods. I will be referring to the code I have written, which is included in the Appendix. You can also find the link to the original GitHub code there. Also, although I have done my best to format the code in a somewhat pretty way, please excuse the sometimes iffy display from the `listings` package in LaTeX.

Matrix Inversion Problem

Verify SOR solves $A\vec{x} = \vec{b}$

We are given the SOR equation

$$\vec{x}^{(n+1)} = (D + \epsilon L)^{-1}[\epsilon\vec{b} - (\epsilon U + (\epsilon - 1)D)\vec{x}^{(n)}]$$

Where $A = L + D + U$. Rearranging this we get

$$\begin{aligned}
&\vec{x}^{(n+1)} = (D + \epsilon L)^{-1}[\epsilon\vec{b} - (\epsilon U + (\epsilon - 1)D)\vec{x}^{(n)}] \\
&(D + \epsilon L)\vec{x}^{(n+1)} = \epsilon\vec{b} - (\epsilon U + (\epsilon - 1)D)\vec{x}^{(n)} \\
&D\vec{x}^{(n+1)} - D\vec{x}^{(n)} = \epsilon\vec{b} - (\epsilon U + \epsilon D)\vec{x}^{(n)} - \epsilon L\vec{x}^{(n+1)}
\end{aligned}$$

Now, at convergence, we know that $\vec{x}^{(n+1)} = \vec{x}^{(n)} = \vec{x}^{*}$, so

$$\begin{aligned}
&D\vec{x}^{(n+1)} - D\vec{x}^{(n)} = \epsilon\vec{b} - (\epsilon U + \epsilon D)\vec{x}^{(n)} - \epsilon L\vec{x}^{(n+1)} \\
&D\vec{x}^{*} - D\vec{x}^{*} = 0 = \epsilon\vec{b} - (\epsilon U + \epsilon D)\vec{x}^{*} - \epsilon L\vec{x}^{*} \\
&(\epsilon U + \epsilon D)\vec{x}^{*} + \epsilon L\vec{x}^{*} = \epsilon\vec{b} \\
&\epsilon(L + D + U)\vec{x}^{*} = \epsilon\vec{b} \\
&A\vec{x}^{*} = \vec{b}
\end{aligned}$$

Derivation of matrix B

We are given

$$A = \begin{pmatrix} 6 & -2 \\ -2 & 4 \end{pmatrix}$$

Which gives

$$L = \begin{pmatrix} 0 & 0 \\ -2 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 6 & 0 \\ 0 & 4 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & -2 \\ 0 & 0 \end{pmatrix}$$

Now, from $\vec{x}^{(n+1)} = \vec{c} - B\vec{x}^{(n)}$ and our original SOR equation, we can deduce that

$$\vec{c} = (D + \epsilon L)^{-1}(\epsilon\vec{b}), \quad B = (D + \epsilon L)^{-1}(\epsilon U + (\epsilon - 1)D)$$

To write this in terms of ϵ, find $(D + \epsilon L)^{-1}$:

$$\begin{aligned}
D + \epsilon L &= \begin{pmatrix} 6 & 0 \\ 0 & 4 \end{pmatrix} + \epsilon \begin{pmatrix} 0 & 0 \\ -2 & 0 \end{pmatrix} = \begin{pmatrix} 6 & 0 \\ -2\epsilon & 4 \end{pmatrix} \\
(D + \epsilon L)^{-1} &= \frac{1}{(6 \times 4) - (0 \times -2\epsilon)} \begin{pmatrix} 4 & 0 \\ 2\epsilon & 6 \end{pmatrix} = \begin{pmatrix} \frac{1}{6} & 0 \\ \frac{\epsilon}{12} & \frac{1}{4} \end{pmatrix}
\end{aligned}$$

Compute $\epsilon U + (\epsilon - 1)D$:

$$\epsilon U + (\epsilon - 1)D = \epsilon \begin{pmatrix} 0 & -2 \\ 0 & 0 \end{pmatrix} + (\epsilon - 1) \begin{pmatrix} 6 & 0 \\ 0 & 4 \end{pmatrix}$$
$$\epsilon U + (\epsilon - 1)D = \begin{pmatrix} 6(\epsilon - 1) & -2\epsilon \\ 0 & 4(\epsilon - 1) \end{pmatrix}$$

Now, find B:

$$B = (D + \epsilon L)^{-1}(\epsilon U + (\epsilon - 1)D)$$
$$B = \begin{pmatrix} \frac{1}{6} & 0 \\ \frac{\epsilon}{12} & \frac{1}{4} \end{pmatrix} \begin{pmatrix} 6(\epsilon - 1) & -2\epsilon \\ 0 & 4(\epsilon - 1) \end{pmatrix}$$
$$B = \begin{pmatrix} (\epsilon - 1) & -\frac{\epsilon}{3} \\ \frac{\epsilon}{2}(\epsilon - 1) & -\frac{\epsilon^2}{6} + (\epsilon - 1) \end{pmatrix}$$

Spectral radius method to rank choices of ϵ

Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of matrix B. The spectral radius of B is then defined as

$$\rho(B) = \max\{|\lambda_1|, \ldots, |\lambda_n|\}$$

So, for each of the ϵ given, we can find the spectral radius ρ of B:

- $\epsilon = 1$ (Gauss-Seiel case):

$$B = \begin{pmatrix} 0 & -\frac{1}{3} \\ 0 & -\frac{1}{6} \end{pmatrix} \implies \lambda_1 = 0, \lambda_2 = -\frac{1}{6} \implies \rho(B) = \frac{1}{6} \approx 0.1667$$

- $\epsilon = 0.97$ (under-relaxation):

$$B = \begin{pmatrix} -0.03 & -0.32333 \\ -0.01455 & -0.18682 \end{pmatrix} \implies \rho(B) = 0.2126$$

- $\epsilon = 1.03$ (over-relaxation):

$$B = \begin{pmatrix} 0.03 & -0.34333 \\ 0.01545 & -0.14682 \end{pmatrix} \implies \rho(B) = 0.1085$$

Now, since a system will converge faster with smaller eigenvalues, we can see that over-relaxation gives the fastest convergence.

SOR introduces a variable ϵ that allows us to tune B and hence minimise $\rho(B)$ to allow for faster convergence. In the example above, we can see that a small change in ϵ from 1 to 1.03 caused a 54% increase in convergence rate. Therefore, I would conclude that SOR is an improvement over the Gauss-Seidel method.

Optimisation

In this question I was asked to modify code to add tolerance to it. I then was asked to add the golden-search method to find the minimum and evaluate the log value of the error at each iteration. You can find the code in under Appendix: Optimisation.

Producing these graphs:

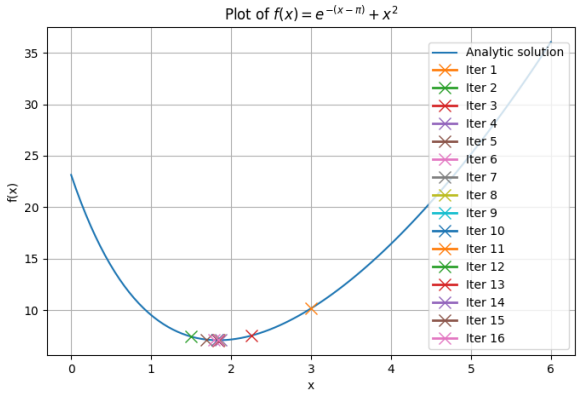


Figure 1: Plot of $f(x) = e^{-(x-\pi)} + x^2$.

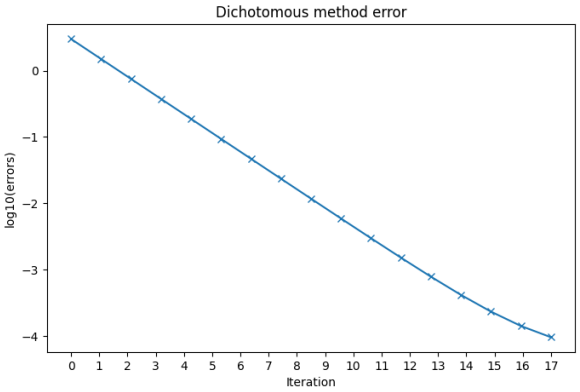


Figure 2: Logarithmic dichotomous error against iteration step.

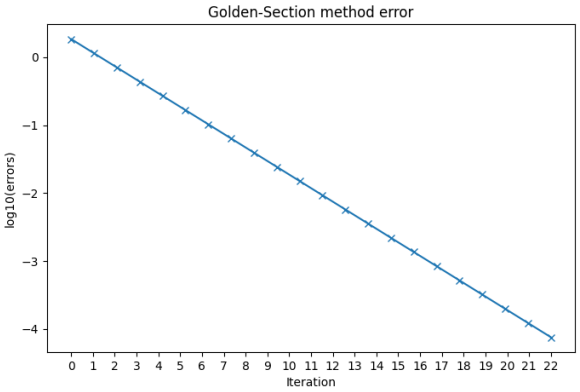


Figure 3: Logarithmic golden-section error against iteration step.

The minimum occurs at $x \approx 1.839$.

As you can see, the error decreases quicker when using the dichotomous method, meeting the tolerance in 17 steps opposed to 22 steps. This is an expected result as dichotomous error halves each iteration and the golden-search roughly decreases by a factor of 0.618 (1 over the golden ratio).

Further analysis

However, with further analysis of more factors, such as wall time and function calls, we get the following graphs:

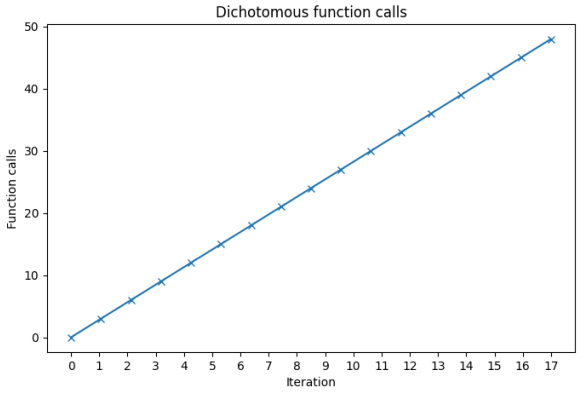


Figure 4: Total dichotomous function calls for current iteration.

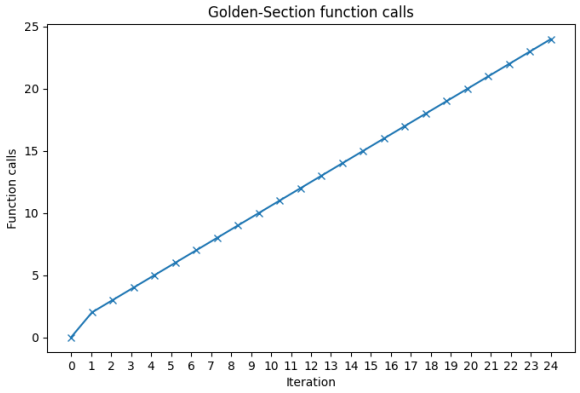


Figure 5: Total golden-section function calls for current iteration.

The linear nature of these plots allows us to estimate the number of function calls per iteration rather easily. We calculate that, on average, the dichotomous method has $2.8 \approx 3$ function calls per iteration and the golden-search method has $0.97 \approx 1$ function call per iteration. This is a significant improvement in performance and is evident when measuring the wall time of each method: the dichotomous program runs for 6.4 milliseconds and the golden-search method runs for 0.2 milliseconds ($\approx 32\times$ faster).

As an experienced programmer, these results are not surprising as function calls are far more expensive than inline arithmetic calculations for CPUs and therefore minimising function calls should be prioritised over the number of iterations when designing algorithms. If required, function-dominated behaviour could be achieved by using `time.sleep()` to introduce an artificial delay within the function being minimised, but I don't recommend this for large numbers of iterations as programs could take a long time to run.

If you're interested to know more, please read Appendix: Why are function calls expensive? for more information on this topic.

Multi-component non-linear ODEs

I was asked to write code to solve the following three-dimensional set of non-linear ODEs:

$$\begin{aligned}\frac{dx}{dt} &= -ax^3 + 3byz^4 \\ \frac{dy}{dt} &= ax^3 - byz^2 \\ \frac{dz}{dt} &= -byz^2 + c\sin x\end{aligned}$$

The code can be found under Appendix: Multi-component non-linear ODEs and can be adjusted for different intial conditions, a, b, c variables, timestep, and max iterations.

But, with the conditions given in the brief:

$$\begin{aligned}&a = 1, \quad b = -1.5, \quad c = 0.2 \\ &x(0) = 1, \quad y(0) = 0, \quad z(0) = 0.5 \\ &\Delta t = 0.001, \quad t_{max} = 20\end{aligned}$$

We get the following graphs:

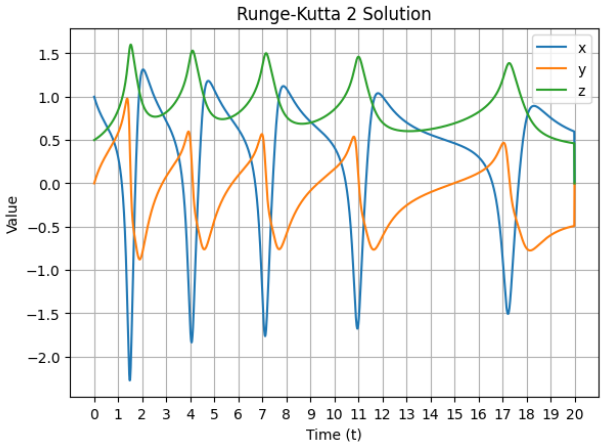


Figure 1: Runge-Kutta 2 x, y, and z values against time t.

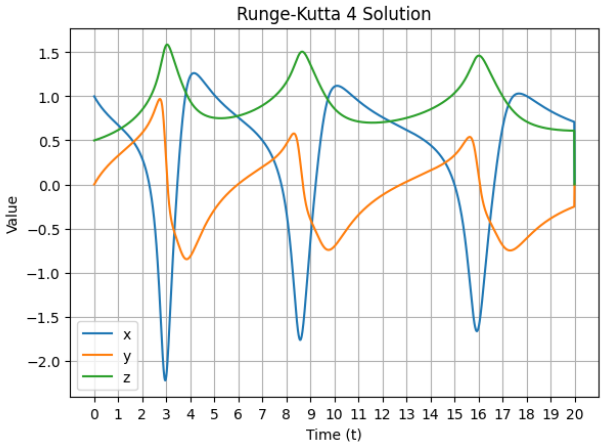


Figure 2: Runge-Kutta 4 x, y, and z values against time t.

Implicit non-linear ODE

We are given the non-linear ODE

$$\frac{dx}{dt} = \sin(x^3)$$

Let $f(x) = \sin(x^3)$ and with $\Delta t = 0.1$ gives. Then, with Backward-Euler x_{n+1} becomes

$$\begin{aligned} x_{n+1} &= x_n + \Delta t f(x_{n+1}) \\ x_{n+1} &= x_n + \Delta t \sin(x_{n+1}^3) \end{aligned}$$

Since x_{n+1} is within the sin component of the equation, we cannot rearrange this so that x_{n+1} is the subject, hence the method is implicit and instead must be solved using

$$g(x_{n+1}) = x_{n+1} - x_n - \Delta t \sin(x_{n+1}^3) = 0$$

This is a non-linear equation and hence we need to iterate using the Newton-Raphson method to solve $g(x_{n+1}) = 0$:

$$\begin{aligned} &z^{(k+1)} = z^{(k)} - \frac{g(z^{(k)})}{g'(z^{(k)})} \quad \text{with our initial guess } z^{(0)} = x_n. \\ &g(z) = z - x_n - \Delta t \sin(z^3) \\ &\implies g'(z) = 1 - 3z^2 \Delta t \cos(z^3) \end{aligned}$$

Now, using the code I wrote under Appendix: Implicit non-linear ODE to solve for x with initial conditions $x(0) = 1$ on interval $t \in [0, 5]$ with $\Delta t = 0.1$ gives:

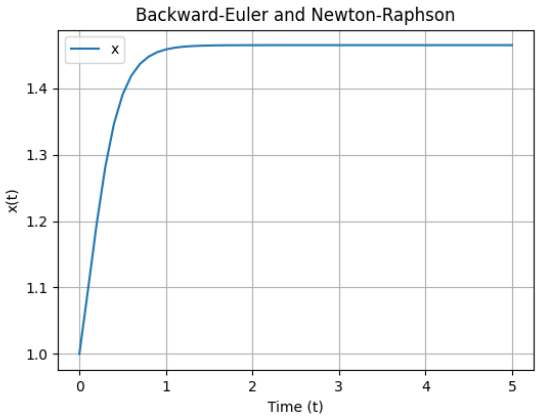


Figure 1: Plot of x solutions against t.

Appendix

6.1 Code

The original GitHub project is linked here.

Optimisation

Tolerance and Golden-Search with error evaluation

```
import numpy as np
import matplotlib.pyplot as plt
import time

golden_ratio = (np.sqrt(5) - 1) / 2 # Define the golden ratio

# User-defined function
fdich = lambda x: np.exp(-x + np.pi) + np.pow(x, 2)

# Initial values
a = 0.0
b = 6.0
niter = 40
tolerance = 1e-4

# Plot analytic function
x = np.linspace(a, b, int(1/0.01))  # about 100 steps across interval
y = fdich(x)

plt.figure(figsize=(8, 5))
plt.plot(x, y, label="Analytic solution")
plt.grid(True)

colors = plt.cm.tab20(np.linspace(0, 1, niter))
legend_entries = ["Analytic solution"]

def golden_section_search(a, b, tol=1e-4): # Define the golden-search algorithm
    start_time = time.time()
    c = b - golden_ratio * (b - a)  # Set c to the left bracket offset by the difference multiplied
        by the golden ratio.
    d = a + golden_ratio * (b - a)  # Set d to right bracket ""
    fc, fd = fdich(c), fdich(d)
    totalFunctionCalls = 2
    functionCalls = [0, totalFunctionCalls]
    err = np.abs(b - a) / 2
    errors = []
    while err > tol: # Check if the difference between the two brackets is less than the tolerance.
                                                                    "".
        if fc < fd: # If f(c) < f(d) then set the right bracket to d.
            b, d, fd = d, c, fc
            # Choose new interval [a, d].
            # Point d now becomes c.
            # Reuse f(c).
            c = b - golden_ratio * (b - a) # Calculate new point c.
            fc = fdich(c) # Find f(c).
        else: # Else set the left bracket to c. Can change fdich(c) < fdich(d) to fdich(c) > fdich(d
            ) for maximum.
            a, c, fc = c, d, fd
            # Choose new interval [c, b].
            # Point c now becomes d.
            # Reuse f(d).
            d = a + golden_ratio * (b - a) # Calculate new point d.
            fd = fdich(d) # Find f(d).
        totalFunctionCalls += 1
        err = np.abs(b - a) / 2
        errors.append(err)
```

```
        functionCalls.append(totalFunctionCalls)
    end_time = time.time()
    total_time = end_time - start_time
    return total_time, (b + a) / 2, errors, functionCalls # When the brackets are within a tolerance
        of each other return the midpoint of the two.

def iteration(a, b, tol=1e-4):
    iter_count = 0
    err = abs(b - a) / 2
    eps = tol / 2
    totalFunctionCalls = 0

    print(" iter, a, b, err")
    start_time = time.time()

    errors = [err]
    functionCalls = [totalFunctionCalls]

    while err > tol:
        x1 = a + 0.5 * (b - a) - eps
        x2 = a + 0.5 * (b - a) + eps

        u = fdich(x1)
        v = fdich(x2)

        midpoint = (a + b) / 2
        fmid = fdich(midpoint)

        totalFunctionCalls += 3
        functionCalls.append(totalFunctionCalls)

        # Plot midpoint marker
        plt.plot(
            midpoint,
            fmid,
            marker="x",
            markersize=10,
            linewidth=2,
            # color=colors[iter_count]
        )
        legend_entries.append(f"Iter {iter_count + 1}")

        if u > v:
            a = x1
        elif u < v:
            b = x2
        else:
            print("help")

        err = abs(b - a) / 2

        errors.append(err)
        print(iter_count, a, b, err)
        iter_count += 1

    end_time = time.time()
    total_time = end_time-start_time

    plt.title("Plot of $f(x) = e^{-(x-\pi)}+x^{2}$")
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.legend(legend_entries, loc="best")
    plt.show()

    return total_time, errors, functionCalls

def plot_errors(errors, title):
    plt.figure(figsize=(8, 5))
    x = np.linspace(0, len(errors), len(errors))
    plt.plot(x, np.log10(errors), marker="x")
    plt.xticks(range(0, len(errors) + 1))
    plt.xlabel("Iteration")
    plt.ylabel("log10(errors)")
    plt.title(title)
    plt.show()

def plot_function_calls(function_calls, title):
```

```
    plt.figure(figsize=(8, 5))
    x = np.linspace(0, len(function_calls), len(function_calls))
    plt.plot(x, function_calls, marker="x")
    plt.xticks(range(0, len(function_calls) + 1))
    plt.xlabel("Iteration")
    plt.ylabel("Function calls")
    plt.title(title)
    plt.show()

    slope, intercept = np.polyfit(x, function_calls, 1)
    return {
        'slope': slope,
        'intercept': intercept,
    }

dichotomous_time, dichotomous_errors, dichotomous_function_calls = iteration(a, b, tolerance)
golden_time, xmin, golden_errors, golden_function_calls = golden_section_search(a, b)

plot_errors(dichotomous_errors, "Dichotomous method error")
plot_errors(golden_errors, "Golden-Section method error")

dichotomous_analysis = plot_function_calls(dichotomous_function_calls, "Dichotomous function calls")
golden_analysis = plot_function_calls(golden_function_calls, "Golden-Section function calls")

print(f'xmin = {xmin:0.4f}')
print(f'Dichotomous: time = {dichotomous_time:0.4f}, slope = {dichotomous_analysis['slope']:0.4f}')
print(f'Golden: time = {golden_time:0.4f}, slope = {golden_analysis['slope']:0.4f}')
```

Multi-component non-linear ODEs

```
import numpy as np
import matplotlib.pyplot as plt

a, b, c = 1, -1.5, 0.2

def F(t, X):
    x, y, z = X # Deconstruct X into components.
    return np.array( # Construct a nparray with new values from system (dx/dt, dy/dt, dz/dt).
        [
            (-a * np.pow(x, 3)) + (3 * b * y * np.pow(z, 4)),
            (a * np.pow(x, 3)) - (b * y * np.pow(z, 2)),
            (-b * y * np.pow(z, 2)) + (c * np.sin(x)),
        ]
    )

def rungeKutta2Step(t, X, dt): # Runge-Kutta 2
    k1 = F(t, X)
    k2 = F(t + dt, X + (k1 * dt))
    return X + ((k1 + k2) * dt * 0.5) # New value of X.

def rungeKutta4Step(t, X, dt): # Runge-Kutta 4
    k1 = F(t, X)
    k2 = F(t + (dt * 0.5), X + (k1 * dt * 0.5))
    k3 = F(t + (dt * 0.5), X + (k2 * dt * 0.5))
    k4 = F(t + dt,    X + (k3 * dt))
    return X + ((k1 + (k2 * 0.5) + (k3 * 0.5) + k4) * dt / 6) # New value of X.

def integrate(rungeKuttaMethod, initial_conditions, t_max, dt): # Function to integrate to time
    t_max with dt and initial conditions
    steps = int(t_max / dt) # Total steps in integration.
    X = np.zeros((steps + 1, 3)) # Initialise nparray with columns of all zeros.
    X[0] = initial_conditions # Set initial conditions.
    for t in range(1, steps): # Iterate through each delta step.
        X[t] = rungeKuttaMethod(t, X[t - 1], dt) # Set each t value to the Runge-Kutta returned
            value for the last X value.
    return X # Return each step of X.

def plot(X, t_max, dt, title): # Plot X against time.
    size_t = max(t_max, 18)  # Set minimum plot size.
    plt.figure(figsize=(size_t / 3, (size_t + 5) * 9 / 48)) # Size the plot so x-axis isn't squished
         and adjust height to 16:9 ratio.
    plt.plot(X[:, 0], label="x") # Plot x-values.
    plt.plot(X[:, 1], label="y") # Plot y-values.
    plt.plot(X[:, 2], label="z") # Plot z-values.
    total_ticks = int(t_max + 1) # Total ticks is actual +1 as starts at 0.
    plt.xticks( # Show the x-axis in seconds not dt.
        np.arange(0, total_ticks / dt, 1 / dt),
        range(0, total_ticks, 1),
    )
    plt.xlabel("Time (t)") # Label x-axis.
    plt.ylabel("Value") # Label y-axis.
    plt.title(title) # Title plot.
    plt.legend() # Add legend.
    plt.grid(True) # Show gridlines.
    plt.show() # Display plot.

# Initial conditions & total time to integrate for (along with precision dt).

initial_conditions = [1.0, 0.0, 0.5]
t_max = 20.0
dt = 0.01

# Integrate.

X_RungeKutta2 = integrate(rungeKutta2Step, initial_conditions, t_max, dt)
X_RungeKutta4 = integrate(rungeKutta4Step, initial_conditions, t_max, dt)

# Plot results.

plot(X_RungeKutta2, t_max, dt, "Runge-Kutta 2 Solution")
plot(X_RungeKutta4, t_max, dt, "Runge-Kutta 4 Solution")
```

Implicit non-linear ODE

```python
import numpy as np
import matplotlib.pyplot as plt

f = lambda x: np.sin(np.pow(x, 3)) # f(x) = dx/dt.
fp = lambda x: (3.0 * np.pow(x, 2)) * np.cos(np.pow(x, 3)) # f'(x).

g = lambda a, b, dt: b - a - (dt * f(b)) # g(x_n, x_n+1) = Implicit method g(x_{n+1}) = 0.
gp = lambda a, b, dt: 1.0 - (dt * fp(b)) # g'(x_n, x_n+1).

def newtonRaphson(xn, dt, tolerance=1e-10, iterations=50):
    z = xn
    for i in range(iterations):
        z_new = z - g(xn, z, dt) / gp(xn, z, dt)
        if abs(z_new - z) < tolerance: return z_new
        z = z_new
    raise RuntimeError(f"Newton-Raphson did not converge to a value within {iterations} iterations")

def backwardEuler(initialX, t_max, dt, tolerance=1e-10, iterations=50):
    N = int(t_max / dt)
    x = np.zeros(N + 1)
    x[0] = initialX
    for n in range(N):
        x[n + 1] = newtonRaphson(x[n], dt, tolerance, iterations)
    return x

def plot(x, t_max, dt, title): # Plot x against time.
    size_t = max(t_max, 18) # Set minimum plot size.
    plt.figure(figsize=(size_t / 3, (size_t + 5) * 9 / 48)) # Size the plot so x-axis isn't squished
        and adjust height to 16:9 ratio.
    plt.plot(x, label="x") # Plot x-values.
    total_ticks = int(t_max + 1) # Total ticks is actual +1 as starts at 0.
    plt.xticks( # Show the x-axis in seconds not dt.
        np.arange(0, total_ticks / dt, 1 / dt),
        range(0, total_ticks, 1),
    )
    plt.xlabel("Time (t)") # Label x axis.
    plt.ylabel("x(t)") # Label y axis.
    plt.title(title) # Title plot.
    plt.legend() # Add legend.
    plt.grid(True) # Show gridlines.
    plt.show() # Display plot.

# Initial condition x_0 and t_max (with precision dt).

x_0 = 1.0
t_max = 5.0
dt = 0.1

# Solve with Backward-Euler.

x = backwardEuler(x_0, t_max, dt)

# Plot results.

plot(x, t_max, dt, "Backward-Euler and Newton-Raphson")
```

6.2 Why are function calls expensive?

Python is an interpreted language (the interpreter standardly being CPython), which at runtime must:

- Parse the source code into an abstract syntax tree (AST).
- Compile this into bytecode - these are a set of instructions for the computer to follow.
- Execute that bytecode in an evaluation loop.

The overhead that function calls bring are mostly in the execution stage of runtime. So, let's look at what happens when you call a simple function `y = f(x)` during execution:

- Step 1: Evaluate the callable object `f` (our function object).
  - CPython looks up the name `f` in each of the scopes of the program:
    - **locals** - the variables within the current function, whatever that may be.
    - **enclosing scopes** - if the current function is nested (written inside another function with local variables).
    - **globals** - functions within the top scope (for example, constants defined at the top of the file).
    - **builtins** - functions/global variables like `range`, `max`, `min`, etc.
  - The result from this lookup **must** be a callable object since we are actively calling `f` with `()` parenthesis.
  - This process is also not instant and add a cost to our function call already.
- Step 2: The arguments passed, in this case `x`, must then be evaluated.
  - The expression within the parenthesis must is evaluated from left to right.
  - This could mean more name lookups (like above in Step 1), attribute lookups, or indexing.
  - The resulting argument values are then pushed onto the value stack.
  - This is an expensive part of the execution (evaluation and pushing to the stack).
- Step 3: Building a call signature.
  - `f(x)` - creates a simple array of arguments.
  - `f(x, y=3)` - builds a map of the keyword names (kwargs) to values (args).
  - `f(*arg, **kwargs)` - unpacks mappings.
- Step 4: Actually call the function - this uses a generic call structure.
  - Checks if the object (`f`) is callable (if it is a function).
  - Finds the appropriate entry (where to start execution from) to the function.
  - Fortunately, builtins are specifically made to be fast (using special routing) and use C-based methods, so these are less expensive to call.
- Step 5: Creates a new, rather expensive, stack frame. This is an environment for the function to run within, containing:
  - An instruction pointer - this tracks your position in the bytecode.
  - Local variables - this is for fast lookups described partly in Step 2.
  - Global/builtin references - so the function isn't limited to the local scope.
  - An evaluation stack.
  - Exception handling information.
- Step 6: Binds arguments to parameters.
  - CPython maps these onto the function's parameters:
    - Positional parameters.
    - Keyword-only parameters.
    - Default values.
    - args and kwargs.
  - While also checking for errors:
    - Missing required arguments.
    - Unexpected arguments/keywords.

        * Other general checks.

- Step 7: Finally start to execute the bytecode.
  - Initialisation of local variables.
  - Execution begins at the address of the entry point for `f`.
  - CPython starts a bytecode evaluation loop for this new frame:
    * Fetch the next bytecode instruction.
    * Decode instruction into an opcode.
    * Dispatch (jump) to a C code implementation of that opcode.
    * Manipulate the value stack to reflect result of operation.
    * Increment the instruction pointer and repeat.
- Step 8: Create return value and teardown frame.
  - Produces a return value object if necessary.
  - Removes the frame from the call stack.
  - Restores calling frame back to the current frame.
  - Pushes the return value onto the caller's evaluation stack.
- Step 9: Assign result to `y` - updates the local scope binding (of the calling scope) of `y`.

Comparing with the steps for inline arithmetic, for example with `y = x*x + 2`:

- Fetch next bytecode instruction.
- Lookup `x`, which is found within the **locals** array (this is the first lookup array, so extremely fast), and push the value of `x` onto the value stack.
  `stack = [x]`
- Lookup `x` a second time and push onto the stack.
  `stack = [x, x]`
- Multiply `x * x`.
  - Pop top two items from stack (`x` and `x`).
    `stack = []`
  - Determine the correct multiplication operation for the runtime type of the variables (`int`, `float`, etc.).
  - Dispatch to a C code implementation (multiplication in this case).
  - Push result onto the stack.
    `stack = [x_squared]`
- Load `2` and push onto the stack.
  `stack = [x_squared, 2]`
- Add `x_squared + 2`.
  - Pop top two items from stack (`x_squared` and `2`).
    `stack = []`
  - Determine the correct addition operation for the runtime type of the variables (`int`, `float`, etc.).
  - Dispatch to a C code implementation (addition in this case).
  - Push result onto the stack.
    `stack = [result]`
- Store result into `y`.
  - Pop top item from stack (`result`).
    `stack = []`
  - Store `result` in `y`.

As you can see, inline arithmetic is much cheaper for the CPU as it doesn't require argument evaluation, building call signitures, stack frame creation, binding of arguments, error handling, or creation of return values. These are all relatively expensive operations and should be considered when choosing methods or algorithms when large amounts of data or iterations are required.